

THE-772 · B2B 报价系统原型 — 交付说明与使用文档

THE-772 · B2B Quoting System Prototype — Handoff & Usage Guide

关联 Linear ticket: [THE-772] B2B Quoting System — architecture proposal (Roller Shade + Drapery v1)
Related Linear ticket: [THE-772] *Scoping* — B2B Quoting System architecture proposal (Roller Shade + Drapery v1)

编写人 / Author: Yan Yan · 交付对象 / For: Damon Wang · 日期 / Date: 2026-06-13

0. 一句话总结 / TL;DR

中文： THE-772 是一个**「方案调研（Scoping）」ticket**——它要的交付物是三份 *书面文档*（竞品分析、架构方案、分阶段计划）。本 repo 没有止步于写方案，而是直接做出了一个 **端到端可运行的原型**，把架构方案里的绝大多数设计点都用真实可跑的代码证明了。因此：

- **系统能力维度：远超 ticket 要求**——目录、配置、渲染、自动报价、报价版本、预订单、供应商 Excel、状态机、全链路追踪都已真实实现并可演示。
- **书面交付物维度：仍有缺口**——ticket 字面要求的「myblinds.co 竞品分析」「书面架构提案（含与 Installer Portal 共用/分叉的取舍）」「分阶段实施计划 + 工时估算」这三份文档目前不在 repo 中。
- **管线右半段是「模拟」的（符合 ticket 的 out-of-scope）**——Excel 投递、供应商回传、物流单号接入、给零售商的实时推送，目前都由「Supplier Console」页面人工模拟，没有接真实的邮件/SFTP/承运商 API/推送通道。

EN: THE-772 is a **scoping ticket** — its named deliverables are three *written documents* (competitive read, architecture proposal, phased plan). This repo went beyond writing a proposal and built a **working end-to-end prototype** that proves most of the proposed architecture in runnable code. So:

- **System capability: exceeds the ticket** — catalog, configurator, render, auto-quote, pricing versions, pre-orders, supplier Excel, state machine, and full tracking are all really implemented and demoable.
- **Written deliverables: still a gap** — the three documents the ticket literally asks for (myblinds.co competitive read; written architecture proposal incl. the share-vs-fork decision against the Installer Portal; phased plan with effort estimates) are not in the repo.
- **The right half of the pipeline is simulated (consistent with the ticket's out-of-scope)** — Excel delivery, supplier callback, logistics tracking ingestion, and real-time push to the retailer are all driven manually by the "Supplier Console" page; no real email/SFTP/carrier API/push channel is wired.

1. 需求逐条对照 / Requirements Traceability

图例 / Legend: ✔ 已真实实现 Implemented · ● 已实现但为模拟/桩 Simulated or stubbed · ■ 未交付 Not delivered

1.1 零售商前端 UX / Retailer-facing UX

Ticket 要求 / Requirement	状态	实现位置与说明 / Where & notes
经过策展、且供应链可生产的产品目录 / Curated, supply-chain-producible catalog	✔	<code>lib/catalog-data.ts</code> + <code>app/catalog</code> 。共 12 个产品（卷帘 6 + 窗帘 6），2 条产品线。每个产品卡片展示可选遮光度。12 products across 2 lines; each card surfaces its valid opacities.
变体/组合：花色 + 该花色有效的遮光选项 / Pattern + opacity valid for that pattern	✔	每个产品有 <code>validOpacities</code> 。配置器中不可生产的遮光度置灰禁用（ <code>components/Configurator.tsx</code> ），后端 <code>validateConfig</code> 同样强校验，非法组合 <code>POST /api/price</code> 返回 422。约束是数据而非硬编码。Constraint is data-driven and enforced on both client and server.
选中配置的「在场景中」实时渲染 / In-context render for the selected configuration	●	<code>components/renderers.tsx</code> ：参数化 SVG 场景（卷帘/窗帘/色板），每次改动即时重绘。 这是程序化 SVG，作为真实可视化引擎的占位/接入点 ，非最终渲染产品。Programmatic SVG standing in for the real render engine.
按产品线参数化的尺寸输入（窗帘高/宽、轨道宽等） / Dimension input parameterized per line	✔	<code>dimensionFields</code> ：卷帘 width 30–300 / height 40–350；窗帘 rodWidth 60–600 / height 80–400 cm，含 min/max/step。前端即时校验 + 后端 <code>validateConfig</code> 二次校验。
后端公式自动报价 / Auto-quote by backend formulas	✔	<code>lib/pricing.ts</code> + <code>POST /api/price</code> （配置器内 280ms 防抖调用）。两种定价：卷帘=尺寸分档价目表 <code>roller-grid</code> ，窗帘=用料公式 <code>drapery-formula</code> 。

1.2 预订单 + 供应商履约管线 / Pre-order & supplier fulfillment pipeline

Ticket 要求 / Requirement	状态	实现位置与说明 / Where & notes
零售商确认报价 → 提交预订单 / Confirm quote → submit pre-order	✓	<code>POST /api/quotes/:id/submit</code> : 草稿报价转为预订单, 写入首条 <code>order_events</code> 。前端二次确认弹窗 (<code>components/QuoteActions.tsx</code>) 。
生成供应商格式的 Excel 订单文件 / Generate supplier-format Excel	✓	<code>lib/excel.ts</code> (exceljs) : 中英双语 工作簿, 含表头块 + 每行明细 (含工艺参数: 用布米数、幅数、裁切长度) + 合计 + 「说明 Instructions」分页。 <code>GET /api/orders/:id/excel</code> 下载。
Excel 投递给供应商 / Excel delivered to supplier	●	仅下载 (订单详情页与 Supplier Console 的 .xlsx 链接) 。说明页写「queued for delivery」, 但 没有接真实邮件/SFTP/API 投递 。 Download only; no real transport wired.
供应商回传订单号 + 持续状态更新 / Supplier returns Order No + status updates	●	由 Supplier Console (<code>app/supplier</code>) 人工模拟: <code>POST /api/orders/:id/advance</code> 按动作推进状态, 订单号随机生成 (<code>SZF-xxxxx</code>) 。 Simulated via the Supplier Console.
订单号 + 运单号同步进物流层 / Order No + Tracking No sync into logistics	●	运单号在「发货」动作时随机生成 (<code>SFxxxxxxxxxxxxxx</code>) , 承运商硬编码为「SF Express Intl」。 未接真实承运商 API 。 No real carrier integration; tracking is generated, carrier hardcoded.
实时状态推送回零售商, 直到妥投 / Real-time status pushed to retailer until delivery	●	仅站内可见 : 订单详情时间线、Dashboard 活动流、履约管线计数。靠页面刷新/ <code>router.refresh()</code> 更新, 没有邮件/WebSocket/推送通知 等真实「推送」通道。 In-portal only; no real push channel.

1.3 v1 产品线范围 / v1 product-line scope

Ticket 要求 / Requirement	状态	说明 / Notes
仅 Roller Shade + Drapery / Roller Shade + Drapery only	✓	恰好这两条线。 Exactly these two lines.
其余产品线推迟到后续阶段 / Additional lines deferred	✓	架构是数据驱动的, 新增产品线主要是加配置数据, 无新增线。 Architecture is data-driven; adding a line is mostly config.

1.4 Ticket 明列的「交付物」 / The ticket's named deliverables

Ticket 交付物 / Deliverable	状态	说明 / Notes
① myblinds.co 竞品分析 / Competitive read of myblinds.co		repo 中无此文档。 Not in the repo.
② 书面架构提案 / Written architecture proposal		原型 演示 了架构，但没有书面叙述文档；其中「零售商前端与现有 Installer Portal 共用还是分叉 」这一明确要求 完全未被讨论 。 Prototype demonstrates the architecture, but there is no written narrative; the explicit "share vs. fork the Installer Portal" question is not addressed anywhere.
③ 分阶段实施计划 + 工时估算 / Phased plan with effort estimates		repo 中无此文档。 Not in the repo.
认证 / 入驻 / 计费 (ticket 明确 out-of-scope) / Auth, onboarding, billing (explicitly OOS)	 (正确排除)	单一演示零售商「Harbor & Lane Interiors」，无登录。符合 ticket。 Single demo retailer, no auth — correctly excluded.

2. 缺口清单与建议 / Gaps & Recommendations

2.1 必须补的「书面交付物」 / Written deliverables to add

这是 Damon 打开 ticket 核对清单时最可能在意的部分。 This is what Damon will most likely check against the ticket.

- myblinds.co 竞品分析**：他们做得好的、缺口、我们该采纳 vs 差异化的点。 Competitive read: strengths, gaps, adopt vs. differentiate.
- 架构提案书面稿**，尤其补上 ticket 点名但 repo 未答的一条：**零售商门户前端与现有 Installer Portal 的「共用 vs 分叉」取舍**（共享哪些基础设施/组件，哪些 fork）。其余 7 个架构子项可直接以本原型为证据成文。 Written proposal, especially the unaddressed **share-vs-fork decision against the Installer Portal**. The other 7 architecture sub-topics can be written up using this prototype as evidence.
- 分阶段实施计划 + 粗略工时**：把下方「2.2 模拟项」转成真实集成的里程碑。 Phased plan + rough effort, turning the simulated items below into real-integration milestones.

建议措辞 / Suggested framing: 把本原型定位为「架构提案的可运行佐证 (executable proof of the architecture)」, 三份书面文档围绕它补齐即可。Position the prototype as an *executable proof* of the architecture; the three documents wrap around it.

2.2 管线右半段的真实集成 / Real integrations for the right half of the pipeline

以下当前是模拟, 是后续 Phase 的主要工作量来源 / Currently simulated; the bulk of later-phase effort:

- **Excel 投递通道**: 邮件 / SFTP / 供应商 API 适配器 (取代「仅下载」)。Excel transport (email/SFTP/API adapter).
- **供应商回传接入**: 订单号、生产状态的真实回调或轮询。Real supplier callback/polling for order № and production status.
- **物流接入**: 承运商 API 接运单号与轨迹 (取代随机生成 + 硬编码承运商)。Carrier API for tracking & milestones.
- **零售商推送通道**: 邮件 / 站内通知 / WebSocket 实时推送 (ticket 明确点名的 "Retailer update channel")。Real push channel (email / in-app / WebSocket).
- **渲染引擎对接**: 用真实可视化产品替换程序化 SVG。Swap programmatic SVG for the real render engine.

2.3 本次已修复的代码问题 / Code issues fixed in this pass

以下三处在本次交付中**已修复并验证** / The following were fixed and verified in this handoff:

1. **卷帘定价版本无真实价差 (已修复) / Roller pricing versions had no real delta (fixed)**. 原先 seed 把 2026.1 与 2026.2 写入了同一份 ROLLER_PRICING_V1, 两个版本字节相同, 版本备注「motorized +\$5、blackout 1.28→1.30」名不副实——「历史报价锁定版本」这一卖点演示不出来。现已拆为: 2026.1 = motorized 90 / blackout × 1.28 (旧版), 2026.2 (active) = motorized 95 / blackout × 1.30 (现行)。现行报价数值不变, 但两个版本间产生了与备注一致的真实价差。2026.1 and 2026.2 were seeded with the same config. Split into a genuine old version (motorized 90 / blackout × 1.28) and the active one (95 / × 1.30); current prices unchanged, but the version lock is now observable. 涉及 / Files: lib/catalog-data.ts (新增 ROLLER_PRICING_V2)、lib/db.ts (seed 与历史报价改用 V2)。验证: POST /api/price 对 120×160 blackout+motorized 卷帘 → \$196.40, pin 2026.2。
2. **Configurator.tsx 在 effect 内同步 setState (已修复) / Synchronous setState inside an effect (fixed)**. useEffect(() ⇒ setAdded(null), [config, qty]) 触发 React 反模式告警。改为派生状态: 记录上次加入报价时的配置指纹, 仅当当前配置/数量仍匹配时显示「已加入」确认, 配置一变即自动消失, 无需 effect。Replaced the reset-via-effect with a derived added value keyed on the config snapshot.
3. **Configurator.tsx 用 <a> 做站内跳转 (已修复) / Raw <a> for in-app navigation (fixed)**. 「review quote」链接由 改为 next/link 的 <Link>。

修复后 `npm run lint` 零错误通过; `/api/price`、`/pricing`、`/configure/*` 均 200 实测正常。
After the fixes `npm run lint` passes with zero errors; `/api/price`, `/pricing`, `/configure/*` all verified 200.

3. 如何运行 / How to Run

环境: Windows + Node 24。本机首次安装踩过一个 `better-sqlite3` 原生模块的坑, 下面给的是**实测可跑**的步骤。 Environment: Windows + Node 24. First-time install hits a `better-sqlite3` native-build snag; the steps below are the verified working path.

```
# 1) 安装依赖 (跳过原生编译脚本, 避免缺 Python 报错)
#   Install deps, skipping native build scripts (no Python on the box)
npm install --ignore-scripts

# 2) 取 better-sqlite3 的预编译二进制 (Node 24 = ABI v137), 解压到位
#   Fetch the prebuilt better-sqlite3 binary (Node 24 = ABI v137) and unpack it
node -e "const fs=require('fs');fetch('https://registry.npmirror.com/-/binary/better-sqlite3-tar-xzf bsq.tgz -C node_modules/better-sqlite3 && rm bsq.tgz

# 3) 起开发服务器
#   Start the dev server
npm run dev           # http://localhost:3000
```

- 一劳永逸 / Permanent fix: `npm config set better_sqlite3_binary_host "https://registry.npmirror.com/-/binary/better-sqlite3"`, 之后 `npm install` 会自动走镜像拉预编译包, 无需手动两步。
- SQLite 数据库在**首次请求时自动创建并灌入种子数据** (`data/app.db`, 含 2 条历史订单)。想重置成全新演示状态: 删掉 `data/` 文件夹即可。 The SQLite DB is auto-created and seeded on first request; delete `data/` to reset to a clean demo.
- 另有一个无害告警: 本机用户目录下有个多余的 `C:\Users\PC\package-lock.json` 让 Next.js 误判 workspace 根目录, 不影响运行。 Harmless warning: a stray lockfile in the home dir makes Next.js mis-infer the workspace root; no impact.

4. 演示走查 (约 5 分钟) / Demo Walkthrough (~5 min)

按这个顺序点，正好覆盖 ticket 要求的全链路。Follow this order to cover the whole loop the ticket asks for.

1. **Dashboard** `/` — 账户概览：未结报价、在途预订单、生产中、已妥投；两条产品线入口；履约管线分档计数；供应商/物流推来的「最新动态」流。Account stats, two product-line entries, fulfillment pipeline, live activity feed.
2. **Catalog** `/catalog` — 按产品线筛选。注意每张卡片底部的**可选遮光度标签**：变体约束是数据驱动的。Filter by line; note the valid-opacity chips per card — constraints are data, not code.
3. **Configure** `/configure/rs-botanica` (**或任意产品**) — 这是核心页：The core page:
 - 选色 / 遮光度 / 选项 / 尺寸；**不可生产的遮光度自动置灰**（试试 `Solar Screen 3%` → 仅 Sheer）。Non-producible opacities are auto-disabled (try `Solar Screen 3%` — sheer only).
 - 左侧**在场景渲染随每次改动实时更新**；下方滑杆可预览卷帘升降 / 窗帘开合。The in-context render updates live; the slider previews shade position / panel draw.
 - **自动报价**由后端公式引擎计算，给出完整价格明细 + 工艺参数（用布米数、幅数）。Auto-quote with full breakdown and manufacturing facts.
 - 价格右上角标注 **pricing v{版本号}**——客户端的预览价从不被信任，加入报价时后端会重新计价。Client preview price is never trusted; the server re-prices on add.
4. **加入报价** → **Quotes** `/quotes` — 草稿累积明细；进入某条报价可看汇总（含锁定的定价版本），点「Submit pre-order」转预订单（有二次确认）。Draft accumulates lines; submit converts it to a pre-order.
5. **Pre-Orders** `/orders` → **订单详情** — 6 段状态步进条、履约信息（供应商单号/运单号/承运商/ETA）、**事件时间线**，以及右上角 **↓ Supplier order file (.xlsx)**：下载中英双语的供应商订单文件。Status stepper, fulfillment facts, event timeline, and the bilingual supplier Excel download.
6. **Supplier Console** `/supplier` — **模拟供应商系统 + 物流层**：Acknowledge（发订单号）→ Start production → Ship（发运单号）→ In transit → Deliver。每一次推进都会回写到零售商侧的订单时间线和 Dashboard 动态流。Simulates the supplier + logistics side; each push appears on the retailer's timeline and dashboard.
7. **Pricing Versions** `/pricing` — 版本化的报价公式引擎：卷帘=尺寸分档价目表，窗帘=用料 cut-and-make 公式。报价会**锁定**计价所用的版本，调价不影响历史报价。The versioned formula engine; quotes pin the version they were priced with.

5. 架构速览 (给技术读者) / Architecture at a Glance

- **技术栈 / Stack**：Next.js 16 (App Router, 无 `src/`)、React 19、TypeScript、Tailwind v4。原生包 `better-sqlite3` / `exceljs` 在 `next.config.ts` 标记为 `serverExternalPackages`，不进客户端包。

- **数据流 / Data flow**: 服务端组件直接调 `lib/db.ts` 的查询函数; 客户端组件 (Configurator / QuoteActions / SupplierActions) 通过 `app/api/` 下的路由变更数据。客户端永不被信任定价——`POST /api/quote-items` 入库前服务端重新计价。
- **目录是静态 TS, 不是 DB 行 / Catalog is static TS, not DB rows**: 产品线、产品、颜色、定价配置在 `lib/catalog-data.ts`; 只有定价版本、报价、订单、订单事件落在 SQLite。
- **定价引擎 / Pricing**: `lib/pricing.ts` 纯函数、同构。两种 kind: `roller-grid` 与 `drapery-formula`。定价配置是 `pricing_versions` 表里的版本化行; 每条报价行快照完整 `config` + 计算结果 `computation` + 版本号 —— 这就是「历史报价免疫调价」的实现方式。
- **可生产性是数据 / Producibility is data**: 每个产品的 `validOpacities` 约束配置器, 并由 `validateConfig` 在服务端强制 (非法组合返回 422)。
- **订单状态机 / Order state machine**: `submitted → acknowledged → in_production → shipped → in_transit → delivered`, 经 `POST /api/orders/:id/advance` 推进, 乱序操作返回 409, 每次转移写一条 `order_events` —— 事件即零售商侧的更新通道 (时间线、Dashboard 流)。
- **供应商 Excel / Supplier Excel**: `lib/excel.ts` 用 exceljs 生成中英双语工作簿, 由 `GET /api/orders/:id/excel` 提供下载。
- **渲染 / Renders**: `components/renderers.tsx` 参数化 SVG (卷帘 / 窗帘 / 色板) —— 真实可视化引擎的占位。

更详细的工程约定见仓库内 `CLAUDE.md` 与 `AGENTS.md`。See `CLAUDE.md` and `AGENTS.md` in the repo for engineering conventions.

6. 给 Damon 的一页结论 / One-Page Conclusion for Damon

-  **报价系统的端到端可行性已被一个能跑的原型证明**: 目录 → 配置 → 渲染 → 自动报价 → 报价版本锁定 → 预订单 → 供应商 Excel → 状态机 → 全链路追踪, 全部真实可演示。End-to-end feasibility is proven by a working prototype across the entire loop.
-  **管线右半段 (投递 / 供应商回传 / 物流 / 推送 / 渲染引擎) 是有意为之的模拟**, 是后续实施阶段的主要工作量。The right half of the pipeline is intentionally simulated and is where the real implementation effort lies.
-  **ticket 字面要求的三份书面文档 (竞品分析、书面架构提案含「共用 vs 分叉 Installer Portal」、分阶段计划) 仍需补齐**——建议以本原型为佐证来写。The three written deliverables remain to be produced, best written with this prototype as evidence.

一句话 / In one line: **做出来的比要写的多, 要写的还没写。** We built more than was asked to be written; what was asked to be written isn't written yet.